Avantis - Smart Contracts Interaction Guide

Avantis Protocol Smart Contract Interaction Guide

This guide provides detailed documentation for directly interacting with the Avantis Protocol smart contracts without using the Python SDK. This is intended for developers who prefer to use contracts directly via tools like Ethers.js, web3.py, or Foundry scripts.

Contract Addresses

Mainnet Addresses

```
TradingStorage: 0x8a311D7048c35985aa31C131B9A13e03a5f7422d PairStorage: 0x5db3772136e5557EFE028Db05EE95C84D76faEC4 PairInfos: 0x81F22d0Cc22977c91bEfE648C9fddf1f2bd977e5 PriceAggregator: 0x64e2625621970F8cfA17B294670d61CB883dA511 USDC: 0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913 Trading: 0x44914408af82bC9983bbb330e3578E1105e11d4e Multicall: 0xb7125506Ff25211c4C51DFD8DdED00BE6Fa8Cbf7 Referral: 0x1A110bBA13A1f16cCa4b79758BD39290f29De82D
```

Core Data Structures

Trade Struct

```
struct Trade { address trader; // Trader's wallet address uint256 pairIndex; // Trading pair index uint256 index; // Trade index uint256 initialPosToken; // Initial collateral (6 decimals) uint256 positionSizeUSDC; // Collateral in USDC (6 decimals) uint256 openPrice; // Opening price (10 decimals) bool buy; // True for long, false for short uint256 leverage; // Leverage (10 decimals) uint256 tp; // Take profit price (10 decimals) uint256 sl; // Stop loss price (10 decimals) uint256 timestamp; // Trade timestamp }
```

TradeInfo Struct

```
struct TradeInfo { uint256 openInterestUSDC; // Open interest in USDC (6 decimals) uint256 tpLastUpdated; // Last TP update timestamp uint256 slLastUpdated; // Last SL update timestamp bool beingMarketClosed; // Whether trade is being market closed uint256 lossProtection; // Loss protection tier }
```

OpenLimitOrder Struct

```
struct OpenLimitOrder { address trader; // Trader's wallet address uint
256 pairIndex; // Trading pair index uint256 index; // Order index uint
256 positionSize; // Collateral (6 decimals) bool buy; // True for lon
g, false for short uint256 leverage; // Leverage (10 decimals) uint256
tp; // Take profit price (10 decimals) uint256 sl; // Stop loss price
(10 decimals) uint256 price; // Limit price (10 decimals) uint256 slipp
ageP; // Slippage percentage (10 decimals) uint256 block; // Block numb
er uint256 executionFee; // Execution fee (18 decimals) }
```

Trading Contract Methods

1. openTrade

Opens a new trade position.

Contract: `Trading` (0x44914408af82bC9983bbb330e3578E1105e11d4e)

Function Signature:

```
function openTrade( Trade memory t, uint8 _type, uint256 _slippageP ) e
xternal payable returns (uint256 orderId)
```

Parameters:

- `t` (Trade struct): Complete trade information
- `_type` (uint8): Order type enum
  - `0` : MARKET
  - `1` : STOP_LIMIT
  - `2` : LIMIT
  - `3` : MARKET_ZERO_FEE
- `_slippageP` (uint256): Slippage percentage in basis points (10 decimals)

Value: Must include execution fee (typically ~0.00035 ETH)

Returns: `uint256 orderId` - Unique order identifier

Example (Ethers.js):

```
const tradeData = { trader: "0x...", pairIndex: 0, index: 0, initialPos
Token: ethers.utils.parseUnits("100", 6), // 100 USDC positionSizeUSDC:
ethers.utils.parseUnits("100", 6), openPrice: ethers.utils.parseUnits
("50000", 10), // $50,000 buy: true, // Long position leverage: ethers.
utils.parseUnits("2", 10), // 2x leverage tp: ethers.utils.parseUnits
("55000", 10), // $55,000 TP sl: ethers.utils.parseUnits("45000", 10),
// $45,000 SL timestamp: Math.floor(Date.now() / 1000),}; const slippag
e = ethers.utils.parseUnits("100", 10); // 1% slippage const executionF
ee = ethers.utils.parseEther("0.00035"); await tradingContract.openTrad
e(tradeData, 0, slippage, { value: executionFee, });
```

2. closeTradeMarket

Closes a trade position at market price.

Function Signature:

```
function closeTradeMarket( uint256 _pairIndex, uint256 _index, uint256
_amount ) external payable returns (uint256 orderId)
```

Parameters:

- `_pairIndex` (uint256): Trading pair index
- `_index` (uint256): Trade index to close
- `_amount` (uint256): Amount of collateral to close (6 decimals)

Value: Must include execution fee

Returns: `uint256 orderId` - Unique order identifier

Example:

```
const pairIndex = 0; const tradeIndex = 1; const closeAmount = ethers.u
tils.parseUnits("50", 6); // Close 50 USDC worth const executionFee = e
thers.utils.parseEther("0.00035"); await tradingContract.closeTradeMark
et(pairIndex, tradeIndex, closeAmount, { value: executionFee, });
```

3. updateMargin

Updates the margin of an existing trade.

Function Signature:

```
function updateMargin( uint256 _pairIndex, uint256 _index, uint8 _type,
uint256 _amount, bytes[] calldata priceUpdateData ) external payable re
turns (uint256 orderId)
```

Parameters:

- `_pairIndex` (uint256): Trading pair index
- `_index` (uint256): Trade index
- `_type` (uint8): Update type enum
  - `0` : DEPOSIT
  - `1` : WITHDRAW
- `_amount` (uint256): Amount to add/remove (6 decimals)
- `priceUpdateData` (bytes[]): Price oracle update data

Value: Must include 1 wei for price update

Returns: `uint256 orderId` - Unique order identifier

Example:

```
const pairIndex = 0; const tradeIndex = 1; const updateType = 0; // DEP
OSIT const amount = ethers.utils.parseUnits("10", 6); // Add 10 USDC co
nst priceUpdateData = ["0x..."]; // From price oracle await tradingCont
ract.updateMargin( pairIndex, tradeIndex, updateType, amount, priceUpda
teData, { value: 1 } );
```

4. updateTpAndSl

Updates take profit and stop loss levels for a trade.

Function Signature:

```
function updateTpAndSl( uint256 _pairIndex, uint256 _index, uint256 _ne
wSl, uint256 _newTP, bytes[] calldata priceUpdateData ) external payabl
e returns (uint256 orderId)
```

Parameters:

- `_pairIndex` (uint256): Trading pair index
- `_index` (uint256): Trade index
- `_newSl` (uint256): New stop loss price (10 decimals)
- `_newTP` (uint256): New take profit price (10 decimals)
- `priceUpdateData` (bytes[]): Price oracle update data

Value: Must include 1 wei for price update

Returns: `uint256 orderId` - Unique order identifier

Example:

```
const pairIndex = 0; const tradeIndex = 1; const newSl = ethers.utils.p
arseUnits("45000", 10); // $45,000 SL const newTp = ethers.utils.parseU
nits("55000", 10); // $55,000 TP const priceUpdateData = ["0x..."]; //
From price oracle await tradingContract.updateTpAndSl( pairIndex, trade
Index, newSl, newTp, priceUpdateData, { value: 1 } );
```

5. cancelOpenLimitOrder

Cancels a pending limit order.

Function Signature:

```
function cancelOpenLimitOrder( uint256 _pairIndex, uint256 _index ) ext
ernal
```

Parameters:

- `_pairIndex` (uint256): Trading pair index
- `_index` (uint256): Order index to cancel

Example:

```
const pairIndex = 0; const orderIndex = 1; await tradingContract.cancelOpenLimitOrder(pairIndex, orderIndex);
```

Trading Delegate Contract Methods

1. setDelegate

Sets a delegate wallet that can perform **all trade-related actions** on behalf of the caller.

> 💡 This is a **1:1 relationship**: each wallet can have **at most one** delegate, and each delegate can be associated with **only one** principal wallet. The delegate is just another EOA or contract wallet.

Function Signature:

```
function setDelegate(address delegate) external
```

Parameters:

- `delegate` (address): The wallet to authorize as delegate. Use `address(0)` to revoke (see `removeDelegate` below if you prefer an explicit method).

Example:

```
const delegate = "0xDE1E...GATE"; await tradingContract.setDelegate(delegate);
```

2. removeDelegate

Removes the current delegate for the caller.

Function Signature:

```
function removeDelegate() external;
```

Example:

```
await tradingContract.removeDelegate();
```

---

3. delegations (view mapping)

The contract keeps track of delegate relationships with:

```
mapping(address => address) public delegations;
```

- `delegations[owner]` returns the delegate wallet set by `owner`.
- If no delegate is set, it returns `address(0)`.

Example:

```
const owner = "0x0WN3R..."; // Read delegate for owner const delegate =
await tradingContract.delegations(owner); console.log("Delegate walle
t:", delegate);
```

4. delegatedAction

Executes a delegated action **on behalf of** `trader`.

Supports **all trade-related actions** by forwarding `call_data` via `delegatecall` (e.g., `openTrade`, `updateTpAndSl`, `cancelOpenLimitOrder`, etc.).

Function Signature:

```
function delegatedAction( address trader, bytes calldata call_data ) ex
ternal payable returns (bytes memory);
```

Parameters:

- `trader` (address): The wallet for whom the action is executed.
- `call_data` (bytes): ABI-encoded function data for any trade action.

Value: Include whatever execution fee the underlying action requires.

Returns: `bytes` — raw return data from the delegated call.

If the inner call reverts, the original revert reason is bubbled up.

Example (openTrade via delegate, ethers):

```
const trader = "0x0WN3R..."; const tradeData = {/* your trade params
*/}; const slippage = 50; const callData = tradingContract.interface.en
codeFunctionData("openTrade", [ tradeData, 0, slippage, ]); const execu
tionFee = ethers.utils.parseEther("0.00035"); await tradingContract.del
egatedAction(trader, callData, { value: executionFee });
```

Example (updateTpAndSl via delegate, ethers):

```
const trader = "0x0WN3R..."; const pairIndex = 0; const tradeIndex = 1;
const newSl = ethers.utils.parseUnits("45000", 10); const newTp = ether
s.utils.parseUnits("55000", 10); const priceUpdateData = ["0x..."]; //
oracle update payload const callData = tradingContract.interface.encode
FunctionData("updateTpAndSl", [ pairIndex, tradeIndex, newSl, newTp, pr
iceUpdateData, ]); await tradingContract.delegatedAction(trader, callDa
ta, { value: 1 });
```

TradingStorage Contract Methods

1. openTrades

Retrieves trade information for a specific trader and trade.

Contract: `TradingStorage` (0x8a311D7048c35985aa31C131B9A13e03a5f7422d)

Function Signature:

```
function openTrades( address _trader, uint256 _pairIndex, uint256 _inde
x ) external view returns (Trade memory)
```

Parameters:

- `_trader` (address): Trader's wallet address
- `_pairIndex` (uint256): Trading pair index
- `_index` (uint256): Trade index

Returns: `Trade memory` - Complete trade information

Example:

```
const trader = "0x..."; const pairIndex = 0; const tradeIndex = 1; cons
t trade = await tradingStorageContract.openTrades( trader, pairIndex, t
radeIndex );
```

2. openTradesInfo

Retrieves additional trade information.

Function Signature:

```
function openTradesInfo( address _trader, uint256 _pairIndex, uint256 _
index ) external view returns (TradeInfo memory)
```

Returns: `TradeInfo memory` - Additional trade information

3. openTradesCount

Gets the number of open trades for a trader on a specific pair.

Function Signature:

```
function openTradesCount( address _trader, uint256 _pairIndex ) externa
l view returns (uint256)
```

Returns: `uint256` - Number of open trades

4. getOpenLimitOrder

Retrieves limit order information.

Function Signature:

```
function getOpenLimitOrder( address _trader, uint256 _pairIndex, uint25
6 _index ) external view returns (OpenLimitOrder memory)
```

Returns: `OpenLimitOrder memory` - Complete limit order information

PairInfos Contract Methods

1. lossProtectionTier

Calculates the loss protection tier for a trade.

Contract: `PairInfos` (0x81F22d0Cc22977c91bEfE648C9fddf1f2bd977e5)

Function Signature:

```
function lossProtectionTier( Trade memory _trade, bool _isPnl ) externa
l view returns (uint256)
```

Parameters:

- `_trade` (Trade memory): Trade information
- `_isPnl` (bool): Whether calculating for PnL

Returns: `uint256` - Loss protection tier

2. getPriceImpactSpread

Calculates price impact spread for a position.

Function Signature:

```
function getPriceImpactSpread( uint256 _pairIndex, bool _isLong, uint25
6 _positionSize, bool _isOpen ) external view returns (uint256)
```

Parameters:

- `_pairIndex` (uint256): Trading pair index
- `_isLong` (bool): True for long position
- `_positionSize` (uint256): Position size (6 decimals)
- `_isOpen` (bool): Whether opening or closing

Returns: `uint256` - Price impact spread (10 decimals)

3. getSkewImpactSpread

Calculates skew impact spread for a position.

Function Signature:

```
function getSkewImpactSpread( uint256 _pairIndex, bool _isLong, uint256
_positionSize, bool _isOpen ) external view returns (uint256)
```

Returns: `uint256` - Skew impact spread (10 decimals)

PairStorage Contract Methods

1. lossProtectionMultiplier

Gets the loss protection multiplier for a pair and tier.

Contract: `PairStorage` (0x5db3772136e5557EFE028Db05EE95C84D76faEC4)

Function Signature:

```
function lossProtectionMultiplier( uint256 _pairIndex, uint256 _tier )
external view returns (uint256)
```

Returns: `uint256` - Loss protection multiplier (percentage)

2. pairMaxOI

Gets the maximum open interest for a trading pair.

Function Signature:

```
function pairMaxOI( uint256 _pairIndex ) external view returns (uint25
6)
```

Returns: `uint256` - Maximum open interest (6 decimals)

3. pairSpreadP

Gets the spread percentage for a trading pair.

Function Signature:

```
function pairSpreadP( uint256 _pairIndex ) external view returns (uint2
56)
```

Returns: `uint256` - Spread percentage (10 decimals)

PriceAggregator Contract Methods

1. openFeeP

Calculates the opening fee for a position.

Contract: `PriceAggregator` (0x64e2625621970F8cfA17B294670d61CB883dA511)

Function Signature:

```
function openFeeP( uint256 _pairIndex, uint256 _positionSize, bool _isL
ong ) external view returns (uint256)
```

Parameters:

- `_pairIndex` (uint256): Trading pair index
- `_positionSize` (uint256): Position size (6 decimals)
- `_isLong` (bool): True for long position

Returns: `uint256` - Opening fee percentage (12 decimals)

Referral Contract Methods

1. getTraderReferralInfo

Gets referral information for a trader.

Contract: `Referral` (0x1A110bBA13A1f16cCa4b79758BD39290f29De82D)

Function Signature:

```
function getTraderReferralInfo( address _account ) external view return
s (bytes32, address)
```

Parameters:

- `_account` (address): Trader's wallet address

Returns:

- `bytes32` - Referral code
- `address` - Referrer address

2. referrerTiers

Gets the tier of a referrer.

Function Signature:

```
function referrerTiers( address _account ) external view returns (uint256)
```

Returns: `uint256` - Referrer tier ID

3. tiers

Gets tier information by tier ID.

Function Signature:

```
function tiers( uint256 _tierId ) external view returns (uint256 feeDiscountPct, uint256 refRebatePct)
```

Returns:

- `uint256 feeDiscountPct` - Fee discount percentage
- `uint256 refRebatePct` - Referral rebate percentage

4. traderReferralDiscount

Calculates referral discount for a trader and fee.

Function Signature:

```
function traderReferralDiscount( address _account, uint256 _fee ) external view returns (uint256 traderDiscount, address referrer, uint256 rebateShare)
```

Parameters:

- `_account` (address): Trader's wallet address
- `_fee` (uint256): Original fee amount

Returns:

- `uint256 traderDiscount` - Discounted fee for trader
- `address referrer` - Referrer address

- `uint256 rebateShare` - Rebate share for referrer

5. setTraderReferralCodeByUser

Sets a referral code for the caller.

Function Signature:

```
function setTraderReferralCodeByUser( bytes32 _code ) external
```

Parameters:

- `_code` (bytes32): Referral code to set

Example:

```
const referralCode = ethers.utils.formatBytes32String("AVANTIS"); await
referralContract.setTraderReferralCodeByUser(referralCode);
```

Multicall Contract Methods

1. aggregate

Executes multiple calls in a single transaction.

Contract: `Multicall` (0xb7125506Ff25211c4C51DFD8DdED00BE6Fa8Cbf7)

Function Signature:

```
function aggregate( (address target, bytes callData)[] calldata calls )
external returns (uint256 blockNumber, bytes[] memory returnData)
```

Parameters:

- `calls` (tuple[]): Array of target addresses and call data

Returns:

- `uint256 blockNumber` - Block number when calls were executed

- `bytes[] memory returnData` - Return data from each call

Example:

```
const calls = [ { target: tradingStorageAddress, callData: tradingStora
geContract.interface.encodeFunctionData( "openTrades", [trader, 0, 0]
), }, { target: tradingStorageAddress, callData: tradingStorageContrac
t.interface.encodeFunctionData( "openTrades", [trader, 0, 1] ), }, ]; c
onst [blockNumber, returnData] = await multicallContract.aggregate(call
s);
```